

LAB 1 HOMEWORK REPORT

Course: Database and Web Application Development
Lecturers: Nguyen Thanh Binh, Nguyen Phan Thao Nguyen

Student ID: 25560052 Student Name: Lê Minh Tân Class: CSBU109.R11.KHBC

Github: <https://github.com/onecot/csbul09-labs/tree/main/lab1>

Table of Contents

(Lab 1) Homework 1: Design a Library Management Database	2
(Lab 1) Homework 2: Build and Manage a Blog System NoSQL Database on MongoDB	4
(Lab 1B) Homework 1: Managing Order Transactions with SQL Transaction Integrity.....	8
(Lab 1B) Homework 2: Designing a Mongoose Schema for Course Registration (Student & Course).....	12

LAB 1: DEVELOPMENT TOOLS OVERVIEW, ENVIRONMENT SETUP & SQL DATABASE DESIGN

(Lab 1) Homework 1: Design a Library Management Database

Script to create the **library_db** database with 3 tables (**authors**, **books**, and **borrow_records**) linked by foreign keys.

```
1  -- 1. CREATE the database
2  •CREATE DATABASE IF NOT EXISTS library_db;
3  •USE library_db;
4
5  -- 2. CREATE the authors table
6  ✓ •CREATE TABLE IF NOT EXISTS authors(
7      id INT AUTO_INCREMENT PRIMARY KEY,
8      author_name VARCHAR(100) NOT NULL,
9      nationality VARCHAR(100) NOT NULL
10 );
11
12 -- 3. CREATE the books table
13 ✓ •CREATE TABLE IF NOT EXISTS books(
14     id INT AUTO_INCREMENT PRIMARY KEY,
15     title VARCHAR(100) NOT NULL,
16     author_id INT NOT NULL,
17     category VARCHAR(50),
18     publish_year INT,
19     FOREIGN KEY (author_id) REFERENCES authors(id)
20 );
21
22 -- 4. CREATE the borrow_records table
23 ✓ •CREATE TABLE IF NOT EXISTS borrow_records(
24     id INT AUTO_INCREMENT PRIMARY KEY,
25     book_id INT NOT NULL,
26     borrower_name VARCHAR(100) NOT NULL,
27     borrow_date TIMESTAMP NOT NULL,
28     return_date TIMESTAMP,
29     FOREIGN KEY (book_id) REFERENCES books(id)
30 );
```

Insert 3 sample records into each table.

```
32  -- 5. INSERT 3 sample records int each table
33  -- 5.1 INSERT into authors
34  • INSERT INTO authors(author_name, nationality) VALUES
35    ('William Shakespeare', 'English'),
36    ('Leo Tolstoy', 'Russian'),
37    ('Fyodor Dostoevsky', 'Russian');
38
39  -- 5.2 INSERT into books
40  • INSERT INTO books(title, author_id, category, publish_year) VALUES
41    ('Hamlet', 1, 'Tragic Drama', 1601),
42    ('War and Peace', 2, 'Literary Realism', 1869),
43    ('Crime and Punishment', 3, 'Psychological Realism', 1866);
44
45  -- 5.3 INSERT into borrow_records
46  • INSERT INTO borrow_records(book_id, borrower_name, borrow_date, return_date) VALUES
47    (1, 'Tan', '2026-10-08 14:30:00', NULL),
48    (2, 'Thai', '2026-09-15 09:45:00', '2026-10-02 16:05:00'),
49    (3, 'Long', '2026-10-08 10:23:58', '2026-10-13 08:30:00');
```

JOIN statement across all 3 tables to display: Book Title, Author Name, Borrower Name, and Borrow Date.

```
51  -- 6. JOIN across all 3 tables
52  ✓ • SELECT
53      books.title AS 'Book Title',
54      authors.author_name AS 'Author Name',
55      borrow_records.borrower_name AS 'Borrower Name',
56      borrow_records.borrow_date AS 'Borrow Date'
57  FROM books
58  INNER JOIN authors ON books.author_id = authors.id
59  INNER JOIN borrow_records ON books.id = borrow_records.book_id;
```

Output	Result #9		
Book Title	Author Name	Borrower Name	Borrow Date
Hamlet	William Shakespeare	Tan	10/08/2026
War and Peace	Leo Tolstoy	Thai	09/15/2026
Crime and Punishment	Fyodor Dostoevsky	Long	10/08/2026

(Lab 1) Homework 2: Build and Manage a Blog System NoSQL Database on MongoDB

1. Create **blog_db** database and the posts collection. Insert at least 3 posts.

```
> use blog_db
< switched to db blog_db
> db.posts.insertMany([
  {
    title: "Getting Started with Nodejs Express",
    category: "Tech",
    views: 150,
    author: { name: "Tran Van B", email: "tranvanb@gmail.com" },
    tags: ["nodejs", "express", "backend"],
    comments: [
      { user: "Tan", content: "Very clear tutorial, thanks!", created_at: new Date("2026-09-20") },
      { user: "Long", content: "Waiting for part 2 about JWT", created_at: new Date("2026-09-21") }
    ]
  },
  {
    title: "Understanding Async Await in JavaScript",
    category: "Tech",
    views: 85,
    author: { name: "Le Thi C", email: "lethic@gmail.com" },
    tags: ["nodejs", "javascript", "async"],
    comments: [
      { user: "Thai", content: "Finally understand event loop", created_at: new Date("2026-10-01") }
    ]
  },
  {
    title: "My Trip to Da Lat",
    category: "Travel",
    views: 200,
    author: { name: "Pham Van D", email: "phamvand@gmail.com" },
    tags: ["travel", "dalat", "experience"],
    comments: [
      { user: "Tan", content: "Nice photos", created_at: new Date("2026-10-05") }
    ]
  }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac7bd0caf1414213828c1b5'),
    '1': ObjectId('6ac7bd0caf1414213828c1b6'),
    '2': ObjectId('6ac7bd0caf1414213828c1b7')
  }
}
```

2. **Query 1:** Find posts in the 'Tech' category AND with views of 100 or more

```
> db.posts.find({
  category: "Tech",
  views: { $gte: 100 }
})
< {
  _id: ObjectId('6ac7bd0caf1414213828c1b5'),
  title: 'Getting Started with Nodejs Express',
  category: 'Tech',
  views: 150,
  author: {
    name: 'Tran Van B',
    email: 'tranvanb@gmail.com'
  },
  tags: [
    'nodejs',
    'express',
    'backend'
  ],
  comments: [
    {
      user: 'Tan',
      content: 'Very clear tutorial, thanks!',
      created_at: ISODate('2026-09-20T00:00:00.000Z')
    },
    {
      user: 'Long',
      content: 'Waiting for part 2 about JWT',
      created_at: ISODate('2026-09-21T00:00:00.000Z')
    }
  ]
}
```

3. Query 2: Find all posts tagged with 'nodejs'.

```
> db.posts.find({
  tags: "nodejs"
})
< {
  _id: ObjectId('6ac7bd0caf1414213828c1b5'),
  title: 'Getting Started with Nodejs Express',
  category: 'Tech',
  views: 150,
  author: {
    name: 'Tran Van B',
    email: 'tranvanb@gmail.com'
  },
  tags: [
    'nodejs',
    'express',
    'backend'
  ],
  comments: [
    {
      user: 'Tan',
      content: 'Very clear tutorial, thanks!',
      created_at: ISODate('2026-09-20T00:00:00.000Z')
    },
    {
      user: 'Long',
      content: 'Waiting for part 2 about JWT',
      created_at: ISODate('2026-09-21T00:00:00.000Z')
    }
  ]
}
{
  _id: ObjectId('6ac7bd0caf1414213828c1b6'),
  title: 'Understanding Async Await in JavaScript',
  category: 'Tech',
  views: 85,
  author: {
    name: 'Le Thi C',
    email: 'lethic@gmail.com'
  },
  tags: [
    'nodejs',
    'javascript',
    'async'
  ],
  comments: [
    {
      user: 'Thai',
      content: 'Finally understand event loop',
      created_at: ISODate('2026-10-01T00:00:00.000Z')
    }
  ]
}
```

4. **Update:** Add new comment to comments array.

Choose specific post by title (using the **\$push** operator) and increment its views by count by 1 (using the **\$inc** operator).

```
> db.posts.updateOne(
  { title: "Understanding Async Await in JavaScript" },
  {
    $push: {
      comments: {
        user: "Tan",
        content: "Just tried the examples, works perfectly on Node 20",
        created_at: new Date("2026-10-08")
      }
    },
    $inc: {
      views: 1
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

5. **Aggregation:** Calculate the total number of views (**totalViews**) and the total number of posts (**totalPosts**), grouped by category.

```
> db.posts.aggregate([
  {
    $group: {
      _id: "$category",
      totalViews: { $sum: "$views" },
      totalPosts: { $sum: 1 }
    }
  }
])
< {
  _id: 'Tech',
  totalViews: 236,
  totalPosts: 2
}
{
  _id: 'Travel',
  totalViews: 200,
  totalPosts: 1
}
```

(Lab 1B) Homework 1: Managing Order Transactions with SQL Transaction Integrity

Create `store_transaction_db` database.

```
1 • CREATE DATABASE IF NOT EXISTS store_transaction_db;
```

Output

OK, 1 row affected in 14.374ms

Create 4 tables: customers, products, orders, and order_items.

```
22  async function createTables() {
23      try {
24          console.log("Connecting to MySQL Database...");
25          console.log("\n-> Creating tables...");
26
27
28          // Create the 'customers' table
29          await pool.execute(`
30              CREATE TABLE IF NOT EXISTS customers(
31                  id INT AUTO_INCREMENT PRIMARY KEY,
32                  full_name VARCHAR(100) NOT NULL,
33                  balance DECIMAL(12,2) DEFAULT 0
34              )
35          `);
36
37          // Create the 'products' table
38          await pool.execute(`
39              CREATE TABLE IF NOT EXISTS products(
40                  id INT AUTO_INCREMENT PRIMARY KEY,
41                  product_name VARCHAR(150) NOT NULL,
42                  price DECIMAL(12,2) NOT NULL,
43                  stock INT NOT NULL
44              )
45          `);
46
47          // Create the 'orders' table
48          await pool.execute(`
49              CREATE TABLE IF NOT EXISTS orders(
50                  id INT AUTO_INCREMENT PRIMARY KEY,
51                  customer_id INT NOT NULL,
52                  total_amount DECIMAL(12,2) NOT NULL,
53                  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
54                  FOREIGN KEY (customer_id) REFERENCES customers(id)
55              )
56          `);
57
58          // Create the 'order_items' table
59          await pool.execute(`
60              CREATE TABLE IF NOT EXISTS order_items(
61                  id INT AUTO_INCREMENT PRIMARY KEY,
62                  order_id INT NOT NULL,
63                  product_id INT NOT NULL,
64                  quantity INT NOT NULL,
65                  price DECIMAL(12,2) NOT NULL,
66                  FOREIGN KEY (order_id) REFERENCES orders(id),
67                  FOREIGN KEY (product_id) REFERENCES products(id)
68              )
69          `);
70
71          console.log("-> customers, products, orders, and order_items tables are ready!")
72      } catch(error) {
73          console.error('MySQL error:', error.message);
74      }
75  }
```

Open SQL transaction (**beginTransaction**) and create wrapper for further steps.

```
101 async function transactionWorkflow(customer_id, product_id, quantity) {
102   // Open SQL transaction
103   console.log("\n-> Begin transaction...")
104   const conn = await pool.getConnection();
105   await conn.beginTransaction();
106
107   try {
108     // Step 1. SELECT balance/stock + check
109     // Step 2. UPDATE customers
110     // Step 3. UPDATE products
111     // Step 4. INSERT orders
112     // Step 5. INSERT order_items
113
114     await conn.commit(); // all OK -> save
115     console.log("-> Transaction successful.")
116   } catch(error) {
117     console.error("Transaction failed:", error.message)
118     await conn.rollback(); // any fail -> undo all
119   } finally {
120     conn.release();
121   }
122 }
```

Step 1. Check customer's balance AND product's stock if $\geq$ quantity.

```
108 // Step 1. SELECT balance/stock + check
109 const [customerQuery] = await conn.execute('SELECT balance FROM customers WHERE id=?', [customer_id]);
110 const [productQuery] = await conn.execute('SELECT price, stock FROM products WHERE id=?', [product_id]);
111
112 if(!customerQuery[0]) throw new Error("Customer not found");
113 if(!productQuery[0]) throw new Error("Product not found");
114
115 const balance = customerQuery[0].balance;
116 const price = Number(productQuery[0].price);
117 const totalOrder = price * quantity;
118 const stock = productQuery[0].stock;
119
120 if(quantity > stock) throw new Error("Not enough stock");
121 if(balance < totalOrder) throw new Error("Not enough balance");
```

Step 2 & 3. Deduct the amount from the customer's balance, and the quantity from the product's stock.

```
122 // Step 2. UPDATE customers
123 await conn.execute('UPDATE customers SET balance = balance - ? WHERE id=?', [totalOrder, customer_id]);
124
125 // Step 3. UPDATE products
126 await conn.execute('UPDATE products SET stock = stock - ? WHERE id=?', [quantity, product_id]);
```

Step 4. Insert a new order record into **orders**.

```
129 // Step 4. INSERT orders
130 const [newOrder] = await conn.execute(`
131     INSERT INTO orders(customer_id, total_amount) VALUES (?, ?);
132 `, [customer_id, totalOrder]);
```

Step 5. Insert the corresponding line item(s) into **order_items**

```
136 // Step 5. INSERT order_items
137 await conn.execute(`
138     INSERT INTO order_items(order_id, product_id, quantity, price) VALUES (?, ?, ?, ?);
139 `, [newOrder.insertId, product_id, quantity, price]);
```

(Lab 1B) Homework 2: Designing a Mongoose Schema for Course Registration (Student & Course)

Define **Student** schema.

```
13 const studentSchema = new mongoose.Schema({
14   fullName: {
15     type: String,
16     required: [true, 'Full name is required'],
17     trim: true,
18     minlength: [2, 'Full name must be at least 2 characters long']
19   },
20   email: {
21     type: String,
22     required: [true, 'Email is required'],
23     unique: true,
24     lowercase: true,
25     match: [/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/, 'Invalid email format']
26   },
27   courses: [{
28     type: mongoose.Schema.Types.ObjectId,
29     ref: 'Course'
30   }]
31 }, {
32   timestamps: true
33 });
```

Define **Course** schema.

```
35 const courseSchema = new mongoose.Schema({
36   courseCode: {
37     type: String,
38     required: [true, 'Course code is required'],
39     unique: true,
40     minlength: [4, 'Course code must be at least 4 characters long']
41   },
42   courseName: {
43     type: String,
44     required: [true, 'Course name is required'],
45     minlength: [4, 'Course name must be at least 4 characters long']
46   },
47   maxStudents: {
48     type: Number,
49     required: [true, 'Number of max students is required'],
50     min: [0, 'Number of max students must be non-zero.'],
51   },
52   availableSlots: {
53     type: Number,
54     required: [true, 'Number of available slots is required']
55   },
56   students: [{
57     type: mongoose.Schema.Types.ObjectId,
58     ref: 'Student'
59   }]
60 }, {
61   timestamps: true
62 });
```

Create **Student** and **Course** models.

```
64 // Create Student & Course model
65 const Student = mongoose.model('Student', studentSchema);
66 const Course = mongoose.model('Course', courseSchema);
```

enrollCourse(studentId, courseId) implementation.

```
99 // 1. Enroll in a course
100 async function enrollCourse(studentId, courseId) {
101   try {
102     const student = await Student.findById(studentId);
103     const course = await Course.findById(courseId);
104
105     if(!student) throw new Error('Student not found!');
106     if(!course) throw new Error('Course not found!');
107
108     if(course.students.some(id => id.equals(studentId)))
109       throw new Error(`${student.fullName} already enrolled ${course.courseCode}.`);
110
111     if(course.availableSlots <= 0)
112       throw new Error(`${course.courseCode} slots are not available.`);
113
114     course.students.push(studentId);
115     course.availableSlots -= 1;
116     student.courses.push(courseId);
117
118     // Update to Database
119     await course.save();
120     await student.save();
121
122     console.log(`${student.fullName} enrolled ${course.courseCode} successfully!`);
123   } catch(error) {
124     console.log("⊗ Enroll course operation error:", error.message);
125   }
126 }
```

dropCourse(studentId, courseId) implementation.

```
128 async function dropCourse(studentId, courseId) {
129   try {
130     const student = await Student.findById(studentId);
131     const course = await Course.findById(courseId);
132
133     if(!student) throw new Error('Student not found!');
134     if(!course) throw new Error('Course not found!');
135
136     if(!course.students.some(id => id.equals(studentId)))
137       throw new Error('Student ${student.fullName} did not enroll ${course.courseCode}.');
138
139     course.students = course.students.filter(id => !id.equals(studentId));
140     course.availableSlots += 1;
141     student.courses = student.courses.filter(id => !id.equals(courseId));
142
143     // Update to Database
144     await course.save();
145     await student.save();
146
147     console.log(`${student.fullName} dropped ${course.courseCode} successfully!`);
148   } catch(error) {
149     console.log("⊗ Drop course operation error:", error.message);
150   }
151 }
```

Test implemented functions.

```
153  async function homework2() {
154      try {
155          // Clear old sample
156          await Student.deleteMany({});
157          await Course.deleteMany({});
158
159          // Create students
160          const student1 = await createStudent("Le Minh Tan", "leminhtan@example.com");
161          const student2 = await createStudent("Nguyen Tri Nhan", "nguyentrinhan@example.com");
162          const student3 = await createStudent("Le Phuc Hieu", "lephuchieu@example.com");
163          // console.log("Created students: ", student1, student2, student3);
164
165          // Create courses
166          const course1 = await createCourse("CSBU109", "Backend & Database Development", 30, 30);
167          const course2 = await createCourse("SS009", "Scientific Socialism", 60, 0);
168
169          await enrollCourse(student1._id, course1._id);
170          await enrollCourse(student2._id, course2._id);
171          await dropCourse(student3._id, course1._id);
172          await dropCourse(student1._id, course1._id);
173
174          // enrollCourse();
175      } catch(error) {
176          console.log("⊗ Mongoose operation error:", error.message);
177      } finally {
178          await mongoose.connection.close();
179          console.log("-> Mongoose connection closed.");
180      }
181  }
182
183  homework2();
```

Result:

```
● tan@onecot lab1b_hw2 % node homework_many_to_many.js
  ◇ injected env (1) from .env
  -> Connected to MongoDB successfully via Mongoose ODM!
  Le Minh Tan enrolled CSBU109 successfully!
  ⊗ Enroll course operation error: SS009 slots are not available.
  ⊗ Drop course operation error: Student Le Phuc Hieu did not enroll CSBU109.
  Le Minh Tan dropped CSBU109 successfully!
  -> Mongoose connection closed.
```